

What is Amazon EKS?

What is Amazon EKS?

Amazon Elastic Kubernetes Service (Amazon EKS) is a managed service that you can use to run Kubernetes on AWS without needing to install, operate, and maintain your own Kubernetes control plane or nodes. Kubernetes is an open-source system for automating the deployment, scaling, and management of containerized applications.

Amazon EKS runs Kubernetes control plane instances across multiple Availability Zones to ensure high availability. Amazon EKS automatically detects and replaces unhealthy control plane instances, and it provides automated version upgrades and patching for them.

What is Amazon EKS?

Amazon EKS is integrated with many AWS services to provide scalability and security for your applications, including the following capabilities:

- Amazon ECR for container images
- Elastic Load Balancing for load distribution
- IAM for authentication
- Amazon VPC for isolation

Amazon EKS runs up-to-date versions of the open-source Kubernetes software, so you can use all of the existing plugins and tooling from the Kubernetes community. Applications that are running on Amazon EKS are fully compatible with applications running on any standard Kubernetes environment, no matter whether they're running in on-premises data centers or public clouds. This means that you can easily migrate any standard Kubernetes application to Amazon EKS without any code modification.

Amazon EKS control plane architecture

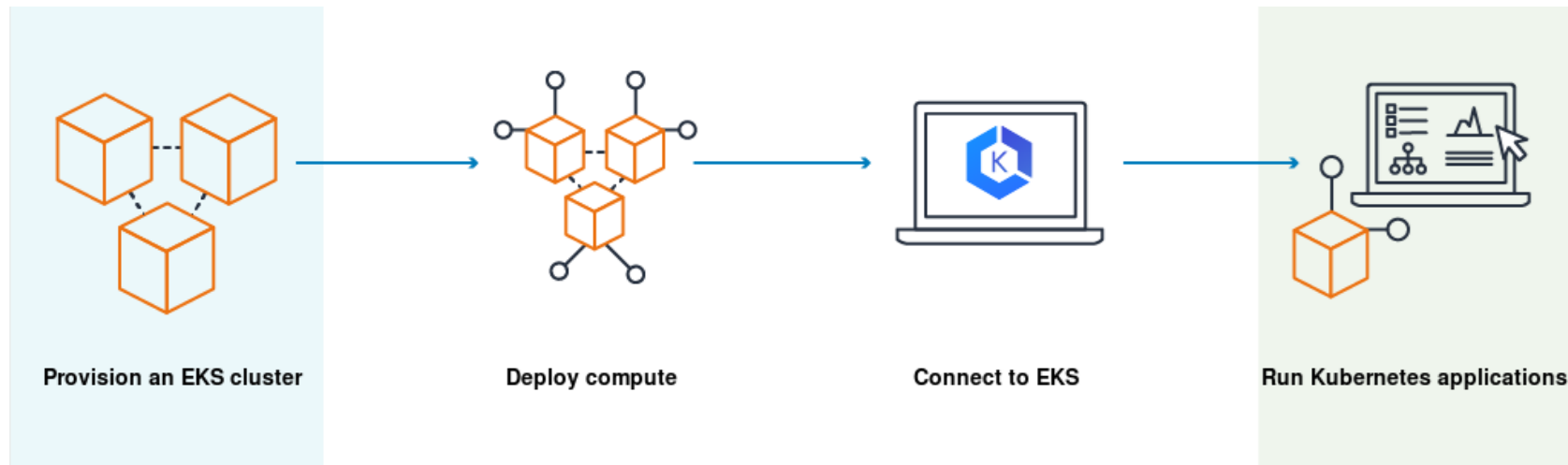
Amazon EKS runs a single tenant Kubernetes control plane for each cluster, and control plane infrastructure is not shared across clusters or AWS accounts.

This control plane consists of at least two API server nodes and three etcd nodes that run across three Availability Zones within a Region. Amazon EKS automatically detects and replaces unhealthy control plane instances, restarting them across the Availability Zones within the Region as needed. Amazon EKS leverages the architecture of AWS Regions in order to maintain high availability. Because of this, Amazon EKS is able to offer an SLA for API server endpoint availability.

Amazon EKS uses Amazon VPC network policies to restrict traffic between control plane components to within a single cluster. Control plane components for a cluster can't view or receive communication from other clusters or other AWS accounts, except as authorized with Kubernetes RBAC policies.

This secure and highly available configuration makes Amazon EKS reliable and recommended for production workloads.

How does Amazon EKS work?



How does Amazon EKS work?

Getting started with Amazon EKS is easy:

1. Create an Amazon EKS cluster in the AWS Management Console or with the AWS CLI or one of the AWS SDKs.
2. Launch managed or self-managed Amazon EC2 nodes, or deploy your workloads to AWS Fargate.
3. When your cluster is ready, you can configure your favorite Kubernetes tools, such as kubectl, to communicate with your cluster.
4. Deploy and manage workloads on your Amazon EKS cluster the same way that you would with any other Kubernetes environment. You can also view information about your workloads using the AWS Management Console.

Pricing

An Amazon EKS cluster consists of a control plane and the Amazon EC2 or AWS Fargate compute that you run pods on. Both Amazon EC2 and Fargate provide:

- **On-Demand Instances** – Pay for the instances that you use by the second, with no long-term commitments or upfront payments.
- **Savings Plans** – You can reduce your costs by making a commitment to a consistent amount of usage, in USD per hour, for a term of 1 or 3 years.

Aligning with Amazon EKS for your self-managed Kubernetes clusters

Amazon EKS Distro is a distribution of the same open-source Kubernetes software and dependencies deployed by Amazon EKS in the cloud. With Amazon EKS Distro, you can create reliable and secure clusters wherever your applications are deployed. You can rely on the same versions of Kubernetes deployed by Amazon EKS, etcd, CoreDNS, upstream CNI, and CSI sidecars with the latest updates, and extended security patching support. Amazon EKS Distro follows the same Kubernetes version release cycle as Amazon EKS and is provided as an open-source project.



Creating an Amazon EKS cluster

Create a cluster with eksctl

This procedure requires eksctl version 0.35.0 or later. You can check your version with the following command:

```
eksctl version
```

To create your cluster with eksctl

1. Create a cluster with the Amazon EKS latest Kubernetes version in your default Region. Replace the <example-values> (including <>) with your own values. You can replace <1.18> with any supported version.

```
eksctl create cluster \  
  --name <my-cluster> \  
  --version <1.18> \  
  --with-oidc \  
  --without-nodegroup
```

Create a cluster with eksctl

Cluster provisioning takes several minutes. During cluster creation, you'll see several lines of output. The last line of output is similar to the following example line.

```
[✓] EKS cluster "<my-cluster>" in "<region-code>" region is ready
```

2. When your cluster is ready, test that your `kubectl` configuration is correct.

```
kubectl get svc
```

Output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
svc/kubernetes	ClusterIP	10.100.0.1	<none>	443/TCP	1m

Create a cluster with eksctl

3. (Optional) If you want to run pods on AWS Fargate in your cluster, then you must [Create a Fargate pod execution role](#) and [Create a Fargate profile for your cluster](#).
4. Follow the procedures in [Launching self-managed Amazon Linux nodes](#) to add Linux nodes to your cluster to support your workloads.
5. (Optional) After you add Linux nodes to your cluster, follow the procedures in [Windows support](#) to add Windows support to your cluster and to add Windows nodes. All Amazon EKS clusters must contain at least one Linux node, even if you only want to run Windows workloads in your cluster.

Create a cluster with the AWS Management Console

Prerequisites

- You have created a VPC and a dedicated security group that meet the requirements for an Amazon EKS cluster.
- You have created an Amazon EKS cluster IAM role to apply to your cluster.

To create your cluster with the console

1. Open the Amazon EKS console at <https://console.aws.amazon.com/eks/home#/clusters>.
2. Choose **Create cluster**.
3. On the **Configure cluster** page, fill in the following fields:
 - **Name** – A unique name for your cluster.
 - **Kubernetes version** – The version of Kubernetes to use for your cluster.
 - **Cluster service role** – Choose the Amazon EKS cluster role to allow the Kubernetes control plane to manage AWS resources on your behalf.
 - **Secrets encryption** – (Optional) Choose to enable envelope encryption of Kubernetes secrets using the AWS Key Management Service (AWS KMS). If you enable envelope encryption, the Kubernetes secrets are encrypted using the customer master key (CMK) that you select. The CMK must be symmetric, created in the same region as the cluster, and if the CMK was created in a different account, the user must have access to the CMK.

To create your cluster with the console

- Kubernetes secrets encryption with an AWS KMS CMK requires Kubernetes version 1.13 or later. If no keys are listed, you must create one first.
 - **Tags** – (Optional) Add any tags to your cluster.
4. Select **Next**.
 5. On the **Specify networking** page, select values for the following fields:
 - **VPC** – Select an existing VPC to use for your cluster. If none are listed, then you need to create one first. For more information, see [Creating a VPC for your Amazon EKS cluster](#).
 - **Subnets** – By default, the available subnets in the VPC specified in the previous field are preselected. Unselect any subnet that you don't want to host cluster resources, such as worker nodes or load balancers. The subnets must meet the requirements for an Amazon EKS cluster.

To create your cluster with the console

- **Security groups** – The SecurityGroups value from the AWS CloudFormation output that you generated when you created your VPC. This security group has ControlPlaneSecurityGroup in the drop-down name.
- (Optional) Choose **Configure Kubernetes Service IP address range** and specify a **Service IPv4 range** if you want to specify which CIDR block Kubernetes assigns service IP addresses from. The CIDR block must meet the following requirements:
 - Within one of the following ranges: 10.0.0.0/8, 172.16.0.0/12, or 192.168.0.0/16.
 - Between /24 and /12.
 - Doesn't overlap with any CIDR block specified in your VPC.

We recommend specifying a CIDR block that doesn't overlap with any other networks that are peered or connected to your VPC. If you don't enable this, Kubernetes assigns service IP addresses from either the 10.100.0.0/16 or 172.20.0.0/16 CIDR blocks.

To create your cluster with the console

- For **Cluster endpoint access** – Choose one of the following options:
 - **Public** – Enables only public access to your cluster's Kubernetes API server endpoint. Kubernetes API requests that originate from outside of your cluster's VPC use the public endpoint. By default, access is allowed from any source IP address. You can optionally restrict access to one or more CIDR ranges such as 192.168.0.0/16, for example, by selecting **Advanced settings** and then selecting **Add source**.
 - **Private** – Enables only private access to your cluster's Kubernetes API server endpoint. Kubernetes API requests that originate from within your cluster's VPC use the private VPC endpoint.
 - **Public and private** – Enables public and private access.

To create your cluster with the console

6. If you selected Kubernetes version 1.17 or earlier on the previous page, skip to the next step. If you selected version 1.18, accept the defaults in the Networking add-ons section to install the latest version of the AWS VPC CNI Amazon EKS add-on. You can only use Amazon EKS add-ons with 1.18 clusters because Amazon EKS add-ons require the Server-side Apply Kubernetes feature, which wasn't available until Kubernetes 1.18. If you selected a different Kubernetes version for your cluster, then this option isn't shown.
7. Select **Next**.
8. On the **Configure logging** page, you can optionally choose which log types that you want to enable. By default, each log type is **Disabled**. For more information, see [Amazon EKS control plane logging](#).
9. Select **Next**.
10. On the **Review and create** page, review the information that you entered or selected on the previous pages. Select **Edit** if you need to make changes to any of your selections. Once you're satisfied with your settings, select **Create**. The **Status** field shows **CREATING** until the cluster provisioning process completes.

To create your cluster with the console

11. (Optional) To use Amazon EKS add-ons, or to enable individual Kubernetes workloads to have specific IAM permissions, you need to enable an OpenID Connect (OIDC) provider for your cluster.
12. Now that you have created your cluster, follow the procedures in [Installing aws-iam-authenticator](#) and [Create a kubeconfig for Amazon EKS](#) to enable communication with your new cluster.
13. After you enable communication, follow the procedures in [Launching self-managed Amazon Linux nodes](#) to add Linux worker nodes to your cluster to support your workloads.
14. (Optional) After you add Linux worker nodes to your cluster, follow the procedures in [Windows support](#) to add Windows support to your cluster and to add Windows worker nodes. All Amazon EKS clusters must contain at least one Linux worker node, even if you only want to run Windows workloads in your cluster.

Create a cluster with the AWS CLI

To create your cluster with the AWS CLI

1. Create your cluster with the following command. Replace the Amazon Resource Name (ARN) of your Amazon EKS cluster IAM role that you created in Amazon EKS cluster IAM role and the subnet and security group IDs for the VPC that you created in Creating a VPC for your Amazon EKS cluster. Replace `<my-cluster>` with your cluster name and `<region-code>` with a supported Region. You can replace `<1.18>` with any supported version.

```
aws eks create-cluster \
  --region <region-code> \
  --name <my-cluster> \
  --kubernetes-version <1.18> \
  --role-arn <arn:aws:iam::111122223333:role/eks-service-role-AWSServiceRoleForAmazonEKS-EXAMPLEBKZRQR> \
  --resources-vpc-config subnetIds=<subnet-a9189fe2>,<subnet-50432629>,securityGroupIds=<sg-f5c54184>
```

Create a cluster with the AWS CLI

Output:

```
{
  "cluster": {
    "name": "<my-cluster>",
    "arn": "arn:aws:eks:<region-code>:<111122223333>:cluster/<my-cluster>",
    "createdAt": <1527785885.159>,
    "version": "<1.18>",
    "roleArn": "arn:aws:iam::<111122223333>:role/eks-service-role-AWSServiceRoleForAmazonEKS-<AFNL4H8HB71F>",
    "resourcesVpcConfig": {
      "subnetIds": [
        "<subnet-a9189fe2>",
        "<subnet-50432629>"
      ],
      "securityGroupIds": [
        "<sg-f5c54184>"
      ],
      "vpcId": "<vpc-a54041dc>",
      "endpointPublicAccess": true,
      "endpointPrivateAccess": false
    },
    "status": "CREATING",
    "certificateAuthority": {}
  }
}
```

Create a cluster with the AWS CLI

To encrypt the Kubernetes secrets with a customer master key (CMK) from AWS Key Management Service (AWS KMS), first create a CMK using the create-key operation.

```
MY_KEY_ARN=$(aws kms create-key --query KeyMetadata.Arn --output text)
```

Add the `--encryption-config` parameter to the `aws eks create-cluster` command. Encryption of Kubernetes secrets can only be enabled when the cluster is created.

```
--encryption-config  
' [{"resources": ["secrets"], "provider": {"keyArn": "<$MY_KEY_ARN>"} } ] '
```

The `keyArn` member can contain either the alias or ARN of your CMK. The CMK must be symmetric, created in the same Region as the cluster, and if the CMK was created in a different account, the user must have access to the CMK. Kubernetes secrets encryption with an AWS KMS CMK requires Kubernetes version 1.13 or later.

Create a cluster with the AWS CLI

2. Cluster provisioning usually takes between 10 and 15 minutes. You can query the status of your cluster with the following command. When your cluster status is ACTIVE, you can proceed.

```
aws eks --region <region-code> describe-cluster --name <my-cluster> --query "cluster.status"
```

3. When your cluster provisioning is complete, retrieve the endpoint and certificateAuthority.data values with the following commands. You must add these values to your kubectl configuration so that you can communicate with your cluster.

- a. Retrieve the endpoint.

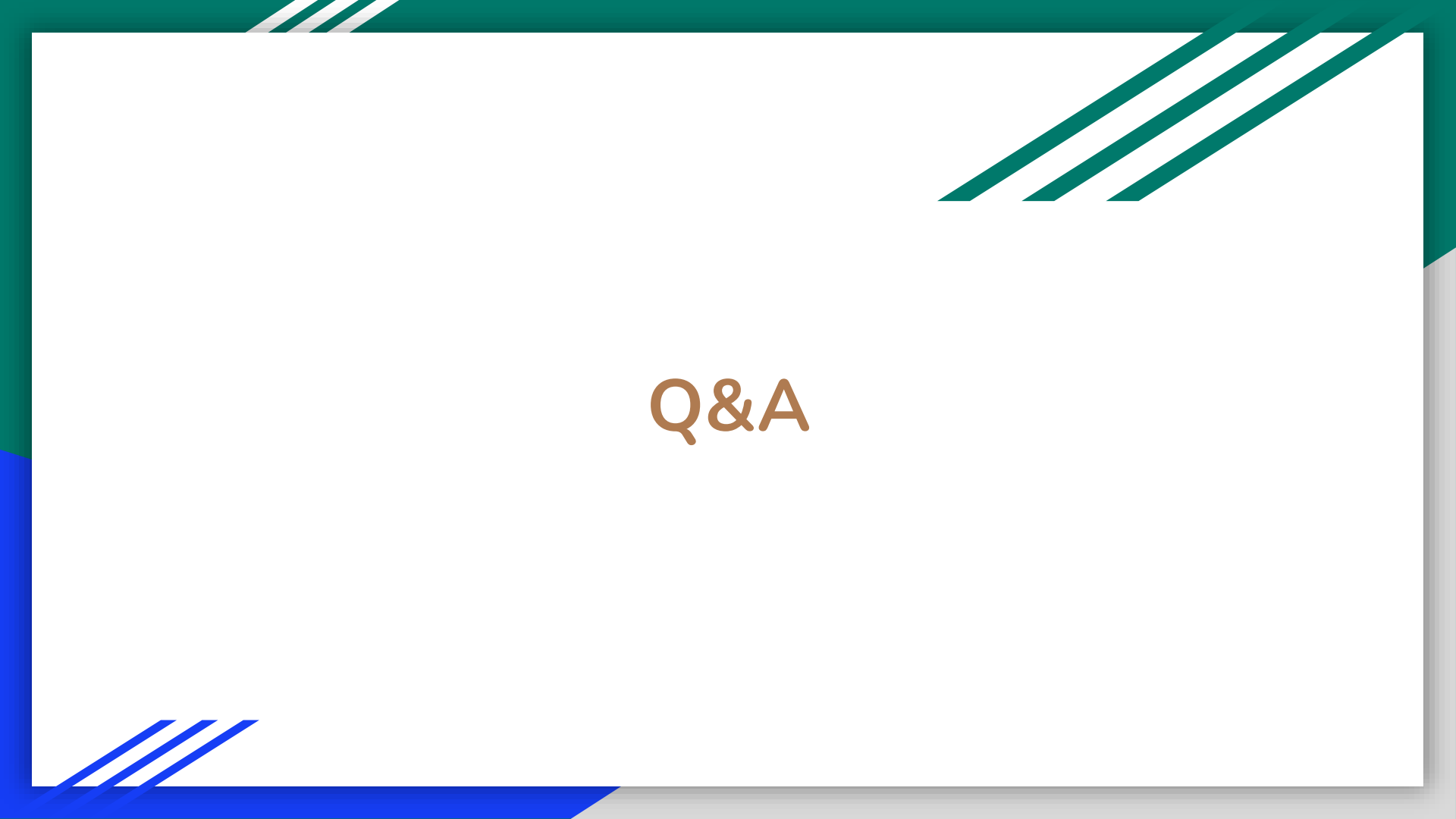
```
aws eks --region <region-code> describe-cluster --name <my-cluster> --query "cluster.endpoint" --output text
```

- b. Retrieve the certificateAuthority.data

```
aws eks --region <region-code> describe-cluster --name <my-cluster> --query "cluster.certificateAuthority.data" --output text
```

Create a cluster with the AWS CLI

4. (Optional) After you add Linux nodes to your cluster, follow the procedures in [Windows support](#) to add Windows support to your cluster and to add Windows nodes. All Amazon EKS clusters must contain at least one Linux node, even if you only want to run Windows workloads in your cluster.
5. (Optional) If the `AmazonEKS_CNI_Policy` managed IAM policy is attached to your node IAM role, we recommend assigning it to an IAM role that you associate to the Kubernetes `aws-node` service account instead.



Q&A